

Final Report

Julian Saria

January 2024

1 Project Definition

The aim of this project is to provide a Web-User Interface for the german male federal division of the sports hockey, table tennis and soccer. It should mainly give the user an overview about three different ratings of the teams, predictions according to these ratings for the next games and an option to evaluate past games on the past predicted ratings. Therefore the two main goal are to predict future games and to evaluate past games to see how good the prediction has been.

2 Use Cases

1. switch between different leagues/sports
2. explore different ratings of clubs
3. make predictions of future games
4. evaluate predictions for past games

3 Dependencies/Technology Stack

3.1 React

React is a JavaScript library for building user interfaces. It allows developers to create reusable UI components and efficiently update the view when the underlying data changes. With React therefore interactive and dynamic user interfaces are possible. Furthermore, components are reusable which enhances code maintainability. And lastly there are states and props which handle the UI logic.

3.2 Node.js

Node.js is a server-side JavaScript runtime that allows the execution of JavaScript code outside of a web browser. It is designed to build scalable network applications. In this project it was used to create a server-side application and APIs using JavaScript. All these requests are asynchronous and node.js handles them efficiently.

3.3 JavaScript

JavaScript is a versatile programming language used for both client-side and server-side development. It is primarily known for enhancing interactivity in web browsers. It was used for writing client-side scripts in React and implementing server-side logic using Node.js.

3.4 Technology Stack

Frontend: React (JavaScript)

Backend: Node.js (JavaScript)

Full Stack Development: JavaScript (React for frontend, Node.js for backend)

This stack provides a unified language (JavaScript) for both client and server, promoting code reuse and streamlining the development process. React ensures a component-based UI structure, while Node.js allows for efficient server-side development.

4 System Design / Architecture

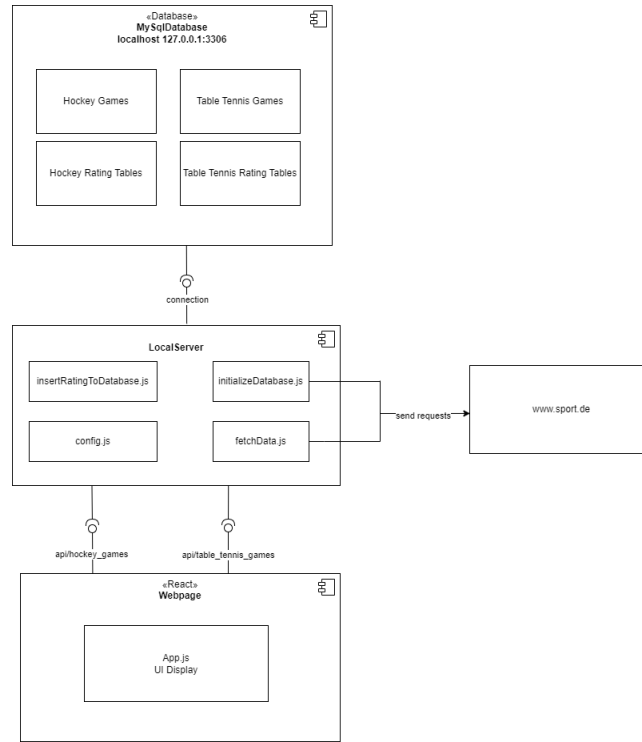


Figure 1: System Architecture

There are three main components. A local MySQL database, a local Server and a React Webpage.

4.1 MySQL database

The MySQL database includes tables from all games over past seasons and the ratings for every team.

4.2 Server

The Server sends requests to the site "www.sport.de" and connects with the database and puts all data into the database while transforming the HTML Code into data that can be put into a MySQL database. Which requests the server sends depends on the data that is in the "config.js" file where all the sites that should be extracted are listed.

4.3 Webpage

The Webpage includes all the UI components, updates them dynamically and also calculates the predictions with the rating that are taken from the database.

5 Rating Algorithms

5.1 Elo Rating Algorithm

The Elo Rating is one of the most common known and used rating Algorithms. It works as follows: Every new team will be assigned with a starting Elo Score. In my Application that is 1500. Now the Elo Scores get adapted by every game over time.

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}$$

E_A ... Expected rating Team A

R_A ... Rating Team A

R_B ... Rating Team B

$$E_B = \frac{1}{1 + 10^{(R_A - R_B)/400}}$$

E_B ... Expected rating Team B

$$R_{A_{new}} = R_A + k * (outcome - E_A)$$

$$R_{B_{new}} = R_B + k * (outcome - E_B)$$

k ... adaptive konstant k

$outcome$... [1, 0.5, 0] = [win, draw, lose]

$R_{A_{new}}$... New Rating Team A

$R_{B_{new}}$... New Rating Team B

That means that the Elo rating just changes with the number of games and is more accurate the more games are in the record.

5.2 Win Percentage Algorithm

In this Algorithm the win rate of a Team is taken over a specific time, for example over three years. But since the win rate might be skewed because a Team just faced weak opponents adaptation rounds are introduced. That means that the win rates get adapted according if they played better or worse teams in the league. A better or worse team is solely determined by their win rate, meaning a team with a higher win rate is seen as a better team. That is why we sum over all the differences of the win rates of the opponent teams. To demonstrate that on an example lets say Team A has a win rate of 50% and played two games. One against $Team_B$ which has a total win rate of 30% and another one against $Team_C$ which has a win rate of 40%. The league average

win rate is always 50 %. So therefore $Team_B$ is 20% below the league average and $Team_C$ is 10% below the league average. We take the mean over all games. In this case two games, so it's 15% below the league average. So that means in the first adjustment round the original win rate from $Team_A$ is lowered by 15% to 35% since $Team_A$ only faced bad teams.

5.3 Points Scored/Game and Points Allowed/Game Algorithm

This Algorithm works similar to the one described above. Over a specific time period all Points Scored and Points Allowed are summed up. Then to get a statically useful measure we divide through the number of games to get an average Points Scored and Points Allowed per game for each Team. Then these Scores are adjusted as in the example explained above, just that this time the league average is the average Scored points per game over all games that are taken into that statistic. That means Teams that scored less points on average than the league average are considered "bad" Teams and Teams that scored more points are considered "good" Teams. Now again adjustment rounds adapt the Scores to make a more realistic ranking.

6 Prediction Algorithms

6.1 Elo Prediction

For the Elo Prediction a sigmoid function is used. This function has a smooth

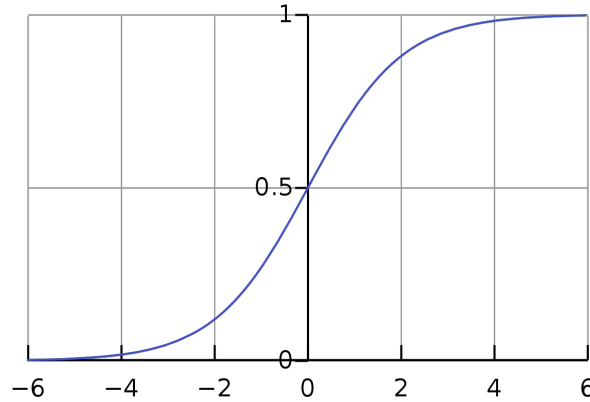


Figure 2: Sigmoid Function

transition from 0 to 1 and can only be between 0 and 1. That is exactly what we need to estimate win percentages. So calculating the win rate of the Teams

happens as followed:

$$sigmoid(x) = \frac{1}{1 + e^{-x}}$$

$$diff_{A-B} = R_A - R_B$$

$$diff_{B-A} = R_B - R_A$$

$$TeamA_{win} = sigmoid(\frac{diff_{A-B}}{spread})$$

$$TeamB_{win} = sigmoid(\frac{diff_{B-A}}{spread})$$

Where the spread is a constant so that the difference between the Elo rating is projected well onto this function. In my program I use 100 so that means if the Elo rating of $Team_A$ is 1700 and the Elo rating of $Team_B$ is 1300, the expected win rate for $Team_A$ is $sigmoid(400/100) = sigmoid(4) = 98\%$ and for $Team_B = 2\%$

6.2 Win Percentage Prediction

To predict the win rates with the win percentages of the Teams, we simply look at the difference of the win rates between two teams. Lets say $Team_A$ has a win rate of 55% and $Team_B$ has a win rate of 48%. The difference is 7%, which then is added to the league average which is 50%. So that means $Team_A$ will have a win rate of 57% and $Team_B$ a win rate of 43%. The only issue with this approach is that if the difference between the win rates of two Teams are ≥ 50 . In this case the win rate for $Team_A$ is 99% and the win rate of $Team_B$ is 1% in my model.

6.3 Points Scored/Game and Points Allowed/Game Prediction

This is the most complex algorithm since it builds upon two important ideas. The Pythagorean Win Expectancy and the Log5 win probability. The Pythagorean Win Expectancy is an estimate of how many games a team should win based upon points scored and points allowed. That there is a correlation between that has been observed over years and proven by multiple studies. The formula here is quite simple:

$$Team_{win} = \frac{PS^{exp}}{PA^{exp} + PS^{exp}}$$

exp ... constant exponent (in my application 2.37)

The exponent depends on the average amount of goals. That means if there are only a few goals per game choose a smaller exponent and vice versa. This way the win probability is calculated. The Log5 win probability estimates the

probability that team A will win a game against team B. It uses that win probabilities of the two Teams that we calculated beforehand with the Pythagorean Win Expectancy and therefore

$$TeamA_{winning} = \frac{TeamA_{win} - (TeamA_{win} * TeamB_{win})}{TeamA_{win} + TeamB_{win} - 2 * TeamA_{win} * TeamB_{win}}$$

$TeamA_{winning}$...The probability that Team A beats Team B
 $TeamA_{win}$...Pythagorean Win Expectancy of Team A
 $TeamB_{win}$...Pythagorean Win Expectancy of Team B

7 Draw

All these predictions have the weakness that they are binary and usually in Sports and also in betting there are three possible results, win, lose and draw. To come up with a good estimate for a draw, we consider the difference between the probability of $Team_A$ and $Team_B$ winning. The higher this difference, the less likely a draw occurs usually. So to calculate the final probabilities of a game the following is done in this project:

$$diff = |TeamA_{win} - TeamB_{win}|$$

$$draw = (1 - diff) * k$$

k ... constant (0.4 in my project)

$$TeamA_{win} = TeamA_{win} - \frac{draw}{2}$$

$$TeamB_{win} = TeamB_{win} - \frac{draw}{2}$$

which are the final results that are displayed in the webpage. This draw calculations are the same for all predictions.

8 Betting Quotes

To now compare these percentages with fair betting Quotes, we just need to

$$TeamA_{Quote} = \frac{1}{TeamA_{win}}$$

$$TeamB_{Quote} = \frac{1}{TeamB_{win}}$$

$$draw_{Quote} = \frac{1}{draw}$$

With these Quotes a betting company wouldn't make any profit. So they usually lower the Quotes so that they are a bit lower than the fair Quotes.

9 Encountered Problems / Limitations

Setting up a dynamic server is quite a lot of work. For now everything needs to be added manually in the files.

10 Reflection / Lesson Learned

Good rating algorithms are quite complex. I am just in the beginning of this topic since all of my algorithms contain quite simple approaches. They still predict quite well but as it is in real life predictions are not always accurate and exceptions can always happen. Regarding the coding part I learned that you have to work very carefully with asynchronous functions that the order is followed where needed and the order is free where not needed.

11 Future Work

Implement other rating algorithms like the Massey rating algorithm or algorithms that use machine learning techniques. Make the Server more dynamic, that means having an UI interface where you can add more sites that can be read and extracted. Furthermore, when starting the server the databases could be updated automatically instead of having to run files additionally.